Essential Math for AI

Contents

Notation (list of symbols)

x	an input example, as a vector (or tensor) of features.

Convention — indexing. Unless a section says otherwise, indices are 1-*based*: a vector $x \in \mathbb{R}^n$ has entries $x_1, \ldots, x_n$, and matrix rows and columns count from 1. Where 0-based indexing is the natural choice (token positions, anything mirroring code), the section says so explicitly.

1 What this chapter covers

This chapter is not all of math — it is exactly the pieces that get used in building models, in the order they are used. It is best learned *in parallel* with the next chapter (*What Learning Means*): start there, and come back here the moment something does not click.

The concepts, in dependency order (★ = load-bearing):

- **Vectors & matrices ★** — the universal data format of ML; a "tensor" is just an n-dimensional array. (§2)
- **Matrix multiplication ★** — a neural net is essentially chained matrix multiplications with nonlinearities in between; know your shapes: $(m \times n) \cdot (n \times p) = (m \times p)$. (§2)
- **Derivatives & partial derivatives** — "how much does the output change if I nudge this input?" (§3)
- **The chain rule ★** — backpropagation *is* the chain rule; the single most load-bearing piece of math on the map. (§4)
- **Gradients** — the vector of all partial derivatives; points toward steepest increase of the loss, and training walks the opposite way. (§5)
- **Probability distributions** — models output distributions (softmax over next tokens), not single answers. (§6)
- **Expectation & variance** — average outcome and its spread; a loss is an expectation over data. (§7)
- **Conditional probability & Bayes** — a language model literally computes $P(\text{next token} \mid \text{previous tokens})$. (§8)
- **Maximum likelihood estimation** — "choose parameters that make the observed data most probable"; minimizing cross-entropy is exactly this. (§9)
- **Entropy, cross-entropy ★, KL divergence** — cross-entropy = how surprised the model is by the true data (the standard loss); KL = distance between distributions (returns in RLHF and VAEs). (§10)

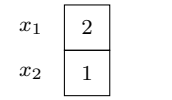
2 Vectors and matrices

Everything a model touches — inputs, parameters, outputs, gradients — is stored one way: as a rectangular array of numbers. This section is about reading those arrays fluently: what they are, how their *shape* is written, and the handful of operations that act on them entry by entry.

2.1 A vector: an ordered list of numbers

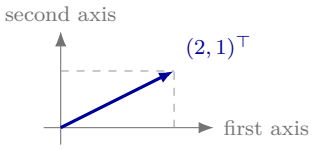
A *vector* is a list of n real numbers in a fixed order. We write $x \in \mathbb{R}^n$ — "x lives in $\mathbb{R}^n$" — meaning x has n real entries; the i-th entry is the *scalar* (single number) x_i. In ML a vector is a *column* by convention, so we mark it $(2,1)^\top$ — the $(\cdot)^\top$ marks that the row as typed stands for a column. (Books often bold vectors, $\mathbf{x}$; we keep plain x and let context and shape annotations carry the type.)

One vector, two views:



$x = (2,1)^\top \in \mathbb{R}^2$

as data: a column of cells

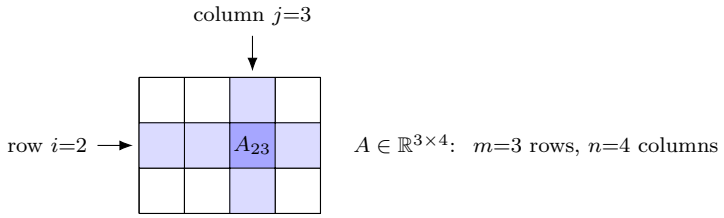


as geometry: an arrow from the origin

Both views matter. Geometry gives intuition (lengths, directions, "nearby" vectors); but in ML the *data* view dominates: a vector is a feature list. A 28×28 grayscale digit, unrolled pixel by pixel, is a vector $x \in \mathbb{R}^{784}$ — the very input of the MNIST collection in this repository.

2.2 A matrix: a grid

A *matrix* is a rectangular grid of numbers. $A \in \mathbb{R}^{m \times n}$ has m *rows* and n *columns* — shape is always read "rows × columns". The entry A_{ij} sits in row i, column j: *row index first.*

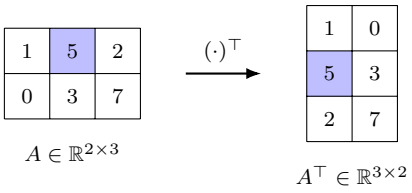


Vectors are just thin matrices: a column vector is $n \times 1$, a row vector is $1 \times n$. That is why the $(\cdot)^\top$ marking matters — $(2,1)^\top$ is 2×1, while a plain $(2,1)$ would be 1×2.

2.3 Transpose: flip the grid

The *transpose* $A^\top$ swaps rows with columns:

$$(A^\top)_{ij} = A_{ji}, \qquad \text{shape } m \times n \;\to\; n \times m.$$



Follow the shaded cell: $A_{12} = 5$ (row 1, column 2) lands at $(A^\top)_{21} = 5$ (row 2, column 1). On vectors, $(\cdot)^\top$ turns a column into a row and back — the same flip.

2.4 Entrywise operations

Vectors and matrices of the *same shape* add entry by entry, and a scalar multiplies every entry:

$$\begin{pmatrix}1\\2\end{pmatrix} + \begin{pmatrix}3\\-1\end{pmatrix} = \begin{pmatrix}4\\1\end{pmatrix}, \qquad 2 \cdot \begin{pmatrix}3\\-1\end{pmatrix} = \begin{pmatrix}6\\-2\end{pmatrix}.$$

There is also an *entrywise product* (the Hadamard product), written $\odot$, which multiplies matching entries — distinct from real matrix multiplication (below):

$$\begin{pmatrix}1\\2\end{pmatrix} \odot \begin{pmatrix}3\\-1\end{pmatrix} = \begin{pmatrix}3\\-2\end{pmatrix}.$$

It looks minor but recurs constantly — masking, gates, dropout all come down to an $\odot$.

A vector's *length* (Euclidean norm) collapses it to one number:

$$\|x\| = \sqrt{x_1^2 + \cdots + x_n^2}, \qquad \|(3,4)^\top\| = \sqrt{9+16} = 5$$

— Pythagoras, in any dimension. Later it will measure quantities such as the size of a gradient.

2.5 The dot product

Multiplication starts with one primitive. The *dot product* of two vectors of the *same* length n multiplies matching entries and sums — many numbers in, *one scalar* out:

$$a \cdot b \;=\; \sum_{i=1}^{n} a_i\, b_i, \qquad (1,2)^\top \cdot (3,-1)^\top = 1\cdot 3 + 2\cdot(-1) = 1.$$

(As matrices it is $a^\top b$: a $1\times n$ row times an $n\times 1$ column gives 1×1 — a scalar, consistent with the shape rule below.) Geometrically it measures *alignment*: large and positive when the vectors point the same way, zero when perpendicular, negative when opposed. That makes the dot product ML's standard measure of *similarity* — the attention scores inside modern language models are exactly dot products.

2.6 Matrix multiplication

The product $C = AB$ is built entirely from dot products:

$$C_{ij} \;=\; \sum_{k=1}^{n} A_{ik}\, B_{kj} \;=\; (\text{row } i \text{ of } A) \cdot (\text{column } j \text{ of } B).$$

Every entry of the result is one dot product — row from the left factor, column from the right.

The shape rule. For those dot products to make sense, the rows of A and the columns of B must have the same length — the *inner* sizes must match, and they disappear; the *outer* sizes survive as the result's shape:

$$(m \times n) \cdot (n \times p) \;=\; m \times p.$$

Predict the output shape *before* computing anything — if the blue inner sizes differ, the product simply does not exist.

Worked, entry by entry (2×3 times 3×2, so the result is 2×2):

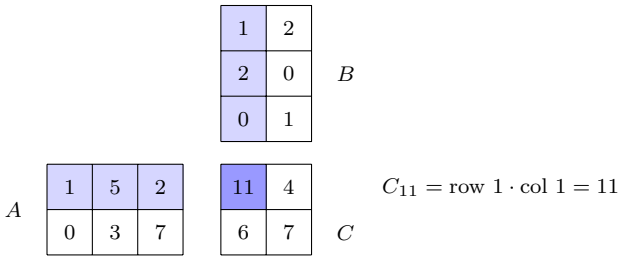
$$\underbrace{\begin{pmatrix} 1 & 5 & 2 \\ 0 & 3 & 7 \end{pmatrix}}_{A\ (2\times3)} \underbrace{\begin{pmatrix} 1 & 2 \\ 2 & 0 \\ 0 & 1 \end{pmatrix}}_{B\ (3\times2)} = \underbrace{\begin{pmatrix} 11 & 4 \\ 6 & 7 \end{pmatrix}}_{C\ (2\times2)},$$

each entry one row-times-column dot product:

$$C_{11} = 1{\cdot}1 + 5{\cdot}2 + 2{\cdot}0 = 11, \qquad C_{12} = 1{\cdot}2 + 5{\cdot}0 + 2{\cdot}1 = 4,$$

$$C_{21} = 0{\cdot}1 + 3{\cdot}2 + 7{\cdot}0 = 6, \qquad C_{22} = 0{\cdot}2 + 3{\cdot}0 + 7{\cdot}1 = 7.$$

The standard picture — B sits above C, A to its left; each cell of C is where a shaded row of A meets a shaded column of B:



Why ML cares. The workhorse case is matrix × vector: with $W \in \mathbb{R}^{m\times n}$ and $x \in \mathbb{R}^n$,

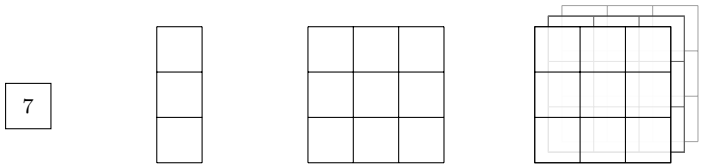
$$y = Wx \in \mathbb{R}^m, \qquad y_i = (\text{row } i \text{ of } W) \cdot x,$$

so a weight matrix is a set of m feature detectors: each output entry scores how strongly the input matches one row. A neural network is essentially this, chained, with nonlinearities in between.

Two warnings. Order matters: $AB \neq BA$ in general (the shapes may not even allow BA). And AB is *not* the entrywise $\odot$ — the two products answer different questions.

2.7 Tensors: the general case

A *tensor* is just an n-dimensional array — the word sounds more advanced than the object is. Each step up adds one axis:



scalar
0-D
shape: —

vector
1-D
shape n

matrix
2-D
shape $m \times n$

3-D tensor
(stacked grids)
shape $c \times m \times n$

Real data comes in exactly these shapes:

Object	Tensor	Shape
one grayscale digit	matrix	28×28
the same, flattened	vector	784
a color image (RGB)	3-D tensor	$3 \times 224 \times 224$
a *batch* of B flat digits	matrix	$B \times 784$
a neural-network layer's weights	matrix	$n_{\text{out}} \times n_{\text{in}}$

The habit to build now: *annotate the shape of everything you write down*, and check that shapes agree before trusting any formula. Shape mismatches are the most common bug in all of ML — and shape reasoning is exactly what made the multiplication rule above mechanical: inner sizes match and vanish, outer sizes survive.

2.8 Symbols

New persistent symbol: x (an input example as a vector) — see the list of symbols. Local to this section:

Local symbol	Meaning (this section)
$\mathbb{R}^n$	the set of vectors with n real entries
x_i	the i-th entry of the vector x (a scalar)
$A \in \mathbb{R}^{m \times n}$	a matrix with m rows and n columns
A_{ij}	the entry of A in row i, column j (row first)
$(\cdot)^\top$	transpose: flip rows and columns; marks column vectors
$\odot$	entrywise (Hadamard) product
$\|x\|$	Euclidean length $\sqrt{\sum_i x_i^2}$
$a \cdot b$	dot product $\sum_i a_i b_i$ — same-length vectors in, one scalar out
$C = AB$	matrix product: $C_{ij} = \sum_k A_{ik} B_{kj}$, row i dot column j
k	the summation index running over the matched inner size
$W,\ y = Wx$	a weight matrix and the vector of its row-dot-products with x
B	batch size (tensor table only — not the matrix B of the product AB)

3 Derivatives and partial derivatives

§2 gave the data format. This section gives the *question* that training asks of every number in a model: **if I nudge this input a little, how much does the output move?** The derivative is that question made precise — and learning is nothing but asking it about every parameter, then nudging each one in the helpful direction.

3.1 The definition: rise over run, shrunk to a point

Take a function f of one number and nudge its input by a small amount h:

$$f'(x) \;=\; \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Read it piece by piece: $f(x+h) - f(x)$ is how much the output moved (the *rise*); h is how much we moved the input (the *run*); the ratio is the output change *per unit* of input change; and the limit $\lim_{h \to 0}$ ("as h shrinks toward 0") makes it exact at the single point x rather than an average over a stretch. Two notations for the same thing: Lagrange's $f'(x)$ and Leibniz's $\frac{df}{dx}$ — we use whichever is clearer.

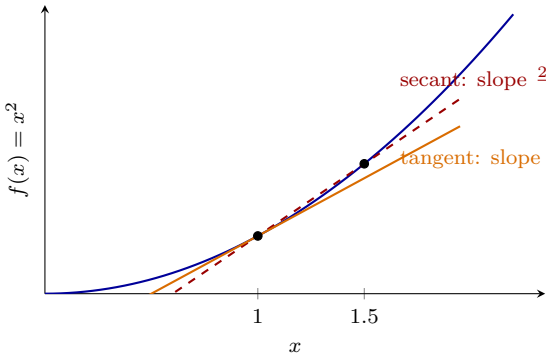
Concrete, with numbers. Take $f(x) = x^2$ at $x = 3$, nudge $h = 0.01$:

$$\frac{f(3.01) - f(3)}{0.01} = \frac{9.0601 - 9}{0.01} = 6.01 \;\approx\; 6 = f'(3),$$

matching the rule $(x^2)' = 2x$ at $x = 3$. Shrink h further and the ratio settles on 6 exactly.

3.2 Seeing it: the tangent line

On a graph, that shrinking ratio is a *secant* line (through two points) collapsing into the *tangent* line (touching at one point). The derivative is the tangent's slope:



The tangent is also the best *local linear stand-in* for f — the approximation everything later leans on:

$$f(x+h) \;\approx\; f(x) + f'(x)\,h \qquad (h \text{ small}).$$

At $x = 3$, $h = 0.01$: predicted $9 + 6 \cdot 0.01 = 9.06$; true 9.0601. One multiplication nearly replaced the whole function.

3.3 The rules you actually use

Derivatives are computed by rules, not by limits, and a handful of rules covers nearly everything:

Rule	Formula	Example
Power	$(x^n)' = n\,x^{n-1}$	$(x^3)' = 3x^2$
Scalar multiple	$(c\,f)' = c\,f'$	$(5x^2)' = 10x$
Sum	$(f+g)' = f' + g'$	$(x^2 + x)' = 2x + 1$
Product	$(f\,g)' = f'\,g + f\,g'$	$(x \cdot x^2)' = x^2 + 2x^2 = 3x^2$

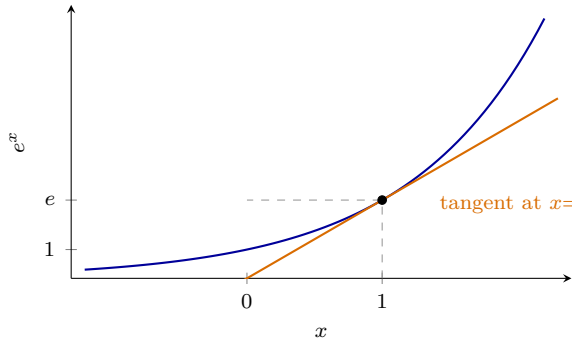
The rule for *composed* functions — $f(g(x))$, one function fed into another — is so load-bearing that it gets the next section to itself: it is the chain rule, and backpropagation is that rule at scale.

3.4 Derivatives you will meet constantly

Beyond powers, a small set of functions does almost all the work in ML — their derivatives are worth memorizing:

$f(x)$	$f'(x)$	Note	Where it shows up in ML
x^n (any real n)	$n\,x^{n-1}$	so $\frac{1}{x} = x^{-1} \to -\frac{1}{x^2}$, $\sqrt{x} = x^{1/2} \to \frac{1}{2\sqrt{x}}$	everywhere
e^x	e^x	its *own* derivative	softmax, sigmoid, attention
$\ln x$	$\frac{1}{x}$	natural log (log base e)	cross-entropy, log-likelihood
$\sin x$	$\cos x$	each is the other's	positional encodings
$\cos x$	$-\sin x$	derivative, one sign flip	
$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)\big(1-\sigma(x)\big)$	derived in the next section	the sigmoid activation in neural networks

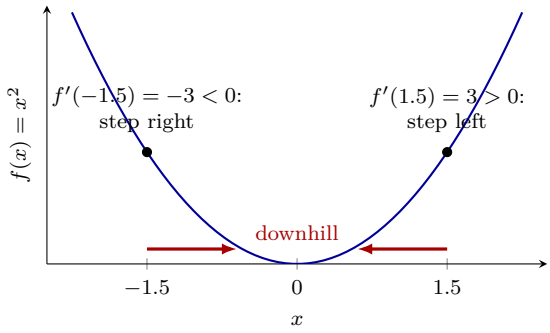
The most important entry is e^x, the exponential with base $e \approx 2.718$: it is the one function whose slope *equals its own height* everywhere — that self-reference is exactly why e (and not 2 or 10) is the "natural" base, and why e^x appears wherever a quantity grows by repeated multiplication:



$\ln x$ is e^x's inverse ("what power of e gives x?"); its derivative $1/x$ is why losses built from logarithms produce such clean gradients. The sigmoid row is a preview: computing it needs the rule for composed functions — next section — and we will derive it there as the first real exercise.

3.5 Which way is downhill

Training never asks only "how steep?" — it asks "*which way do I nudge x to make f smaller?*". The derivative's *sign* answers: positive slope means the function rises to the right, so step left; negative slope, step right. Always *against* the sign. Take again $f(x) = x^2$, so $f'(x) = 2x$: at $x = -1.5$ the slope is $2 \cdot (-1.5) = -3$, at $x = 1.5$ it is $+3$:

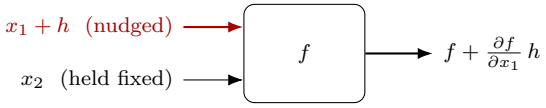


That one sentence — *step against the sign of the derivative* — is gradient descent in one dimension. Training a real model does exactly this, just with millions of dimensions at once.

3.6 Partial derivatives: many inputs, one at a time

Models are functions of *many* inputs. A *partial derivative* asks the same nudge question about one input while **holding all the others fixed**. The symbol changes from d to ∂ ("partial", a curly d), and the definition is unchanged — nudge only x_1:

$$\frac{\partial f}{\partial x_1} = \lim_{h \to 0} \frac{f(x_1 + h,\ x_2) - f(x_1,\ x_2)}{h}.$$



In practice: treat every other variable as a constant and differentiate as usual.

Worked. $f(x_1, x_2) = x_1^2 + 3x_1x_2$:

$$\frac{\partial f}{\partial x_1} = 2x_1 + 3x_2 \quad (x_2 \text{ is a constant here}), \qquad \frac{\partial f}{\partial x_2} = 3x_1 \quad (\text{now } x_1 \text{ is the constant}).$$

At $(x_1, x_2) = (1, 2)$: $\partial f/\partial x_1 = 8$ and $\partial f/\partial x_2 = 3$ — the output moves 8 per unit nudge of x_1, but only 3 per unit nudge of x_2: the partials rank *which inputs matter most at this point.* Collect them all into one vector and you get the *gradient* — §5, where "step against the sign" becomes "step against the vector".

3.7 Why ML cares

A model's loss L is a function of *millions* of inputs — every parameter θ_i is one of them. Training needs exactly the per-parameter sensitivities $\partial L/\partial\theta_i$: which direction, and how strongly, each individual weight should move to make the model less wrong. Derivatives are not a tool ML occasionally uses; they are the entire mechanism of learning. The only real problem left is computing millions of partials *cheaply* — that is what the chain rule (next section) and backpropagation solve.

3.8 Symbols

No new persistent symbols. Local to this section:

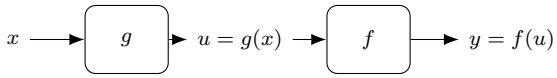
Local symbol	Meaning (this section)
h	a small nudge added to an input
$f'(x) = \frac{df}{dx}$	the derivative: output change per unit input change at x
$\lim_{h \to 0}$	the value the ratio settles on as h shrinks to 0
$\partial f / \partial x_i$	partial derivative: nudge x_i only, all other inputs held fixed
c	a constant (unaffected by the nudge)
e	Euler's number ≈ 2.718: the base whose exponential is its own derivative
$\ln x$	the natural logarithm: the power of e that gives x
$\sigma(x)$	the sigmoid $1/(1 + e^{-x})$ (derivative derived next section)
$f(g(x))$	a composition: g's output fed into f (next section)

4 The chain rule

This is the single most load-bearing piece of math on the whole map. §3 differentiated one function at a time; this section differentiates functions *fed into* functions — and a neural network is nothing but functions fed into functions, dozens of layers deep. Backpropagation — the algorithm that trains neural networks — *is* this rule, applied at scale.

4.1 The setting: composed functions

A *composition* $f(g(x))$ runs x through g, then runs the result through f. Give the intermediate value its own name, u:

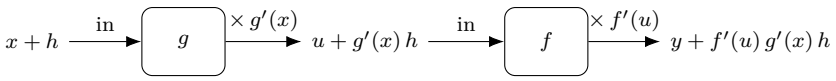


Example: $y = (3x+1)^2$ is the chain $g(x) = 3x+1$ (inner) followed by $f(u) = u^2$ (outer).

4.2 The rule: rates multiply

$$\frac{dy}{dx} \;=\; \frac{dy}{du}\cdot\frac{du}{dx} \qquad \text{equivalently} \qquad \big(f(g(x))\big)' \;=\; f'\big(g(x)\big)\cdot g'(x).$$

The intuition is one sentence: *rates multiply.* If u moves $3\times$ as fast as x, and y moves $2\times$ as fast as u, then y moves $6\times$ as fast as x. Watch a nudge h travel down the chain, scaled by each link's local slope:



The nudge arrives at the output multiplied by $f'(u)\cdot g'(x)$ — and that product is the chain rule. (Each step is just §3's linear stand-in $f(u+k) \approx f(u) + f'(u)\,k$, applied to the nudge each stage hands the next.)

One trap to avoid forever: f' is evaluated **at the inner value** $g(x)$, *not* at x. The outer function never sees x; it only ever sees what g handed it.

4.3 Worked, step by step

Differentiate $y = (3x+1)^2$.

Step 1 — name the pieces. Inner $g(x) = 3x+1$, outer $f(u) = u^2$.

Step 2 — differentiate each piece alone (rules of §3):

$$g'(x) = 3 \quad \text{(sum + scalar rules)}, \qquad f'(u) = 2u \quad \text{(power rule)}.$$

Step 3 — multiply, evaluating f' at the inner value $u = 3x+1$:

$$y' \;=\; f'\big(g(x)\big)\cdot g'(x) \;=\; 2\,(3x+1)\cdot 3 \;=\; 18x+6.$$

Check it two independent ways. Algebraically: expand first, $(3x+1)^2 = 9x^2+6x+1$, whose derivative by the power and sum rules is $18x+6$ — same answer. Numerically at $x=1$: $f(1) = 16$;

nudge $h = 0.01$ gives $(3.03 + 1)^2 = 16.2409$, and

$$\frac{16.2409 - 16}{0.01} = 24.09 \;\approx\; 24 = 18 \cdot 1 + 6.\checkmark$$

4.4 Longer chains: multiply every link

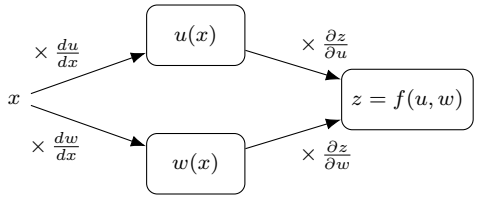
Compositions nest indefinitely, and the rule extends the obvious way — one factor per link:

$$y = f\big(g(h(x))\big) \qquad \Longrightarrow \qquad \frac{dy}{dx} = \frac{dy}{du_2} \cdot \frac{du_2}{du_1} \cdot \frac{du_1}{dx},$$

where $u_1 = h(x)$ and $u_2 = g(u_1)$ name the intermediates. In words: *the derivative of a chain is the product of the local slopes along it.* Notice the bookkeeping advantage: each link only needs its *own* derivative, evaluated at its *own* input — no link needs to know what the rest of the chain looks like. That locality is exactly what backpropagation will exploit: a deep network is one long chain, and the gradient of the loss is a product of cheap local factors collected layer by layer.

4.5 The chain rule with partial derivatives

So far the chain was a single sequence — each value fed exactly one next step. The new case: x reaches the output through *several routes at once.* Say $z = f(u, w)$ where *both* $u = u(x)$ and $w = w(x)$ depend on x:



The rule: **multiply along each path, then add the paths**:

$$\frac{dz}{dx} = \underbrace{\frac{\partial z}{\partial u} \cdot \frac{du}{dx}}_{\text{path via } u} + \underbrace{\frac{\partial z}{\partial w} \cdot \frac{dw}{dx}}_{\text{path via } w}.$$

The partials $\partial z/\partial u$, $\partial z/\partial w$ are §3's one-input-at-a-time sensitivities: each path's multiplier holds the *other* intermediate fixed. The + appears because a nudge of x genuinely arrives at z twice, once through each route: by the linear stand-in, u moves by $\frac{du}{dx}h$ and w moves by $\frac{dw}{dx}h$, and z's response to small moves of its two inputs is the sum of its per-input responses.

Worked. Take $z = u \cdot w$ with $u = x^2$ and $w = 3x$. The partials are $\partial z/\partial u = w$ and $\partial z/\partial w = u$ (differentiate $u\,w$ holding the other factor fixed), so

$$\frac{dz}{dx} = w \cdot 2x + u \cdot 3 = 3x \cdot 2x + x^2 \cdot 3 = 9x^2.$$

Check directly: $z = x^2 \cdot 3x = 3x^3$, whose derivative is $9x^2$. ✓(At $x = 1$: paths give $3 \cdot 2 + 1 \cdot 3 = 9$.) Notice what we just re-derived: with general u, w this *is* the product rule of §3, $(u\,w)' = w\,u' + u\,w'$ — so the product rule is simply the two-path chain rule, written out for a product.

With m routes the sum just grows — $z = f(u_1, \ldots, u_m)$, each u_i depending on x:

$$\frac{dz}{dx} = \sum_{i=1}^{m} \frac{\partial z}{\partial u_i} \cdot \frac{du_i}{dx}.$$

This is the form that runs ML. Inside a network one weight influences the loss through *many* paths — its neuron's output fans out to every neuron of the next layer — and the weight's gradient is exactly this sum over all of them. Backpropagation is the bookkeeping that computes these path-sums for millions of parameters without ever enumerating the paths.

4.6 The payoff: deriving the sigmoid's derivative

§3 promised it; now we can derive it. The sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}} = \big(1+e^{-x}\big)^{-1}$$

is a chain of four small steps: $x \mapsto -x \mapsto e^{-x} \mapsto 1+e^{-x} \mapsto (\,\cdot\,)^{-1}$. Differentiate the inner and outer parts separately, then multiply.

Inner: $u(x) = 1+e^{-x}$. The piece e^{-x} is itself a mini-chain (outer e^{v}, inner $v = -x$ with $v' = -1$), so by the chain rule

$$\frac{du}{dx} = e^{-x}\cdot(-1) = -\,e^{-x}.$$

Outer: $y = u^{-1}$, so by the power rule ($n = -1$)

$$\frac{dy}{du} = -\,u^{-2} = -\frac{1}{(1+e^{-x})^2}.$$

Multiply:

$$\sigma'(x) = \frac{dy}{du}\cdot\frac{du}{dx} = \Big(-\frac{1}{(1+e^{-x})^2}\Big)\cdot\big(-e^{-x}\big) = \frac{e^{-x}}{(1+e^{-x})^2}.$$

Rewrite in terms of σ itself. First note

$$1-\sigma(x) = \frac{(1+e^{-x})-1}{1+e^{-x}} = \frac{e^{-x}}{1+e^{-x}},$$

and therefore

$$\sigma(x)\big(1-\sigma(x)\big) = \frac{1}{1+e^{-x}}\cdot\frac{e^{-x}}{1+e^{-x}} = \frac{e^{-x}}{(1+e^{-x})^2} = \sigma'(x).\checkmark$$

$$\boxed{\sigma'(x) = \sigma(x)\,\big(1-\sigma(x)\big)}$$

This form is why the table in §3 listed it that way: during training you already computed $\sigma(x)$ on the forward pass, so its derivative costs one subtraction and one multiplication — no re-evaluation. Backpropagation relies on exactly this trick.

4.7 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$f(g(x))$	the composition: g runs first, f runs on g's output
$u = g(x)$	the named intermediate value between the links
$f'(g(x))$	the outer derivative, evaluated *at the inner value* (never at x)
u_1, u_2	intermediates of a three-link chain, $u_1 = h(x)$, $u_2 = g(u_1)$
$z = f(u, w)$	an output reached through two routes, $u(x)$ and $w(x)$
$\partial z/\partial u$	one path's multiplier: sensitivity to u with w held fixed
v	inner variable of the mini-chain e^{-x} ($v = -x$)
$\sigma(x)$	the sigmoid $(1 + e^{-x})^{-1}$; $\sigma' = \sigma(1 - \sigma)$

5 Gradients

§3 ended with a promise: collect all the partial derivatives into one vector. This section keeps it. The gradient is that vector, and it answers the question the one-dimensional "step against the sign" could not: *in which direction* should a many-input function be nudged.

5.1 Definition: all the partials, as one vector

The *gradient* of a function f of n inputs is the column vector that collects its n partial derivatives:

$$\nabla f \;=\; \Big(\frac{\partial f}{\partial x_1},\; \frac{\partial f}{\partial x_2},\; \ldots,\; \frac{\partial f}{\partial x_n}\Big)^{\top}.$$

The symbol ∇ ("nabla") is read "the gradient of". Note the shape: f takes a vector of n inputs and returns one number, and ∇f has exactly the input's shape — one sensitivity per input, in the input's own layout.

Worked (the function of §3): $f(x_1, x_2) = x_1^2 + 3x_1x_2$, whose partials were $2x_1 + 3x_2$ and $3x_1$. So

$$\nabla f = \begin{pmatrix} 2x_1 + 3x_2 \\ 3x_1 \end{pmatrix}, \qquad \nabla f(1,2) = \begin{pmatrix} 8 \\ 3 \end{pmatrix}.$$

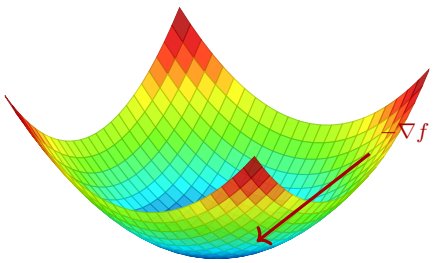
5.2 The direction it points: steepest increase

The gradient is more than a container — it has a geometric meaning. Take a small step δ (a vector: how much each input moves). By §3's linear stand-in, applied to every input at once,

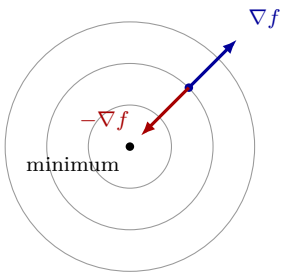
$$f(x + \delta) \;\approx\; f(x) + \nabla f \cdot \delta$$

— the change in f is the *dot product* of the gradient with the step. And §2 said what a dot product measures: *alignment*. Among all steps of the same length, $\nabla f \cdot \delta$ is largest when δ points *along* ∇f, and most negative when δ points *against* it. Therefore:

- ∇f points in the direction of **steepest increase** of f;
- $-\nabla f$ points in the direction of **steepest decrease** — downhill.



the surface $f(x_1, x_2) = x_1^2 + x_2^2$: a bowl



the same bowl seen from above:
the circles are contour lines

Left: the surface itself — a bowl, minimum at the bottom, and $-\nabla f$ leads straight down its wall. Right: the identical bowl viewed from directly above; each circle is a *contour line* (points where f has one constant value), and the bottom of the bowl becomes the center point. At any point, ∇f stands perpendicular to the contour through it, pointing away from the minimum; $-\nabla f$ points toward it. The flat view is how optimization pictures are usually drawn — worth learning to read, because paper is flat and real losses have millions of input axes.

5.3 Step against the vector

§3's one-dimensional rule — step against the *sign* of the derivative — now generalizes to any number of inputs: step against the *vector*. For a loss L with parameters θ (all of them at once, §9):

$$\theta_{\text{new}} \;=\; \theta_{\text{old}} \;-\; \eta\,\nabla L(\theta_{\text{old}}),$$

where η is a small step size. Every parameter moves simultaneously, each against its own partial derivative, and the combined move is the steepest-descent direction of the loss. This update rule *is* gradient descent — the second idea of the spine — and the next chapter is built on it. The gradient ∇L has θ's shape: for a model with millions of parameters, it is a vector of millions of sensitivities, one per parameter, all computed per step — cheaply, by the chain rule's bookkeeping (§4) organized as backpropagation.

5.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
∇f ("nabla")	the gradient: the column vector of all partial derivatives of f
δ	a small step vector: how much each input moves
contour line	a curve along which f keeps one constant value
η	the step size of the update rule (called the *learning rate*)
θ	all model parameters, as in §9

6 Probability distributions

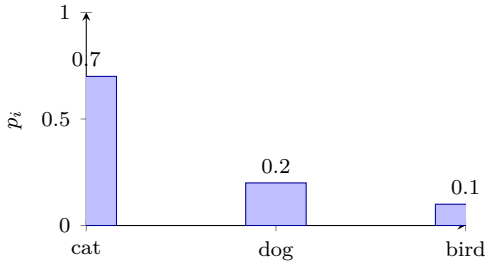
A trained model almost never answers with a single value. A classifier does not say "cat"; it says *cat* 0.7, *dog* 0.2, *bird* 0.1. A language model does not pick the next word; it assigns a probability to *every* word it knows. The object holding those numbers is a *probability distribution*, and this section is about reading and producing them.

6.1 A distribution is a vector with two constraints

Start with the discrete case. A *random variable* X is a quantity whose value is not fixed but drawn from K possible *outcomes*; a distribution assigns each outcome i a probability p_i. Write $P(X = i)$ for "the probability that X turns out to be outcome i". Only two rules:

$$p_i \geq 0 \quad (i = 1, \ldots, K), \qquad \sum_{i=1}^{K} p_i = 1.$$

No probability is negative, and together the outcomes cover everything that can happen. In ML terms (§2): *a discrete distribution over K outcomes is just a vector $p \in \mathbb{R}^K$ whose entries are nonnegative and sum to* 1. A classifier's output layer literally produces this vector:



The heights are the probabilities; they sum to 1 ($0.7 + 0.2 + 0.1$). The bars carry more information than a bare answer: this model favors "cat" but assigns real probability to "dog" — and training will score it on *how much* probability it put on the truth, not just on which bar is tallest.

6.2 The three distributions you will actually meet

Name	Outcomes	Probabilities	Where it appears in ML
Bernoulli	2 (yes/no)	one number p; the other outcome gets $1 - p$	dropout masks, binary labels
Uniform	K, all equal	every $p_i = \frac{1}{K}$	random choice, shuffling
Categorical	K, general	any valid $(p_1, \ldots, p_K)$	classifier and LM outputs

The categorical is the one to remember: every classifier output and every next-token distribution is a categorical — for a modern language model, a categorical over $K \approx 50{,}000$ token outcomes.

6.3 From scores to a distribution: softmax

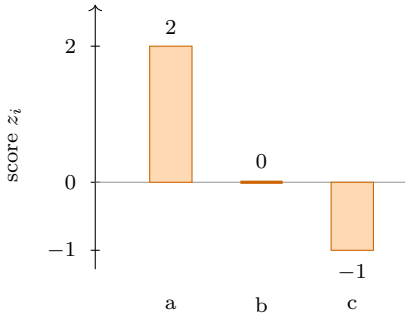
A network's last layer produces K raw scores $z_1, \ldots, z_K$ (the *logits* — just real numbers, possibly negative — not probabilities). *Softmax* converts them into a valid distribution: exponentiate every score — e^{z_i} is always positive, and larger scores stay larger (§3's e^x) — then divide by the total so everything sums to 1:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}.$$

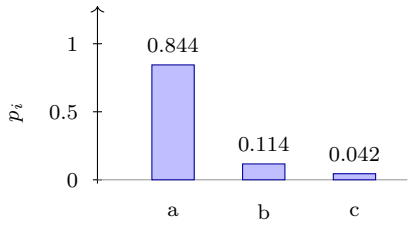
Worked ($K = 3$, scores $z = (2,\ 0,\ -1)^\top$):

$$e^2 \approx 7.389, \quad e^0 = 1, \quad e^{-1} \approx 0.368, \qquad \text{sum} \approx 8.757,$$

$$p \approx \left(\tfrac{7.389}{8.757},\ \tfrac{1}{8.757},\ \tfrac{0.368}{8.757}\right)^\top = (0.844,\ 0.114,\ 0.042)^\top.$$



logits: any real numbers



after softmax: a distribution

Softmax appears at the output of essentially every classifier and language model, and again inside *attention* — the mechanism at the heart of modern language models — where it turns relevance scores into weights. Note what the exponential does: gaps between scores become *ratios* between probabilities, so the largest score dominates without the others dropping to zero.

Why the name "softmax". Compare it with the *hard* maximum-picker, which would answer $(1,\ 0,\ 0)^\top$ — all probability on the largest score, nothing anywhere else. That hard function jumps abruptly when the leading score changes and has derivative 0 everywhere else, so gradient descent (§3) would receive no signal through it. Softmax is the *softened* version of that maximum-picker: our $(0.844,\ 0.114,\ 0.042)^\top$ still clearly favors the winner, but every outcome keeps a nonzero share, and the output moves *smoothly* as the scores move — differentiable everywhere, so it can sit inside a trained network. The name is literal: a **soft max**. And the softness is adjustable by the scores themselves: spread them further apart and softmax approaches the hard version — with $z = (10,\ 0,\ -1)^\top$ it already answers $(0.99994,\ 0.00005,\ 0.00002)^\top$, nearly the hard $(1, 0, 0)^\top$.

6.4 Continuous outcomes: densities and the Gaussian

When outcomes are real numbers (a weight's initial value, a noise sample), single points can no longer carry probability — there are uncountably many of them. Instead a *density* $p(x)$ says how probability is spread out, and only *areas under the curve* are probabilities:

$$P(a \le X \le b) = \int_a^b p(x)\,dx, \qquad \int_{-\infty}^{\infty} p(x)\,dx = 1.$$

The one density to know is the *Gaussian* (or *normal*) $\mathcal{N}(\mu, \sigma^2)$:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

a bell-shaped curve whose two parameters control everything:

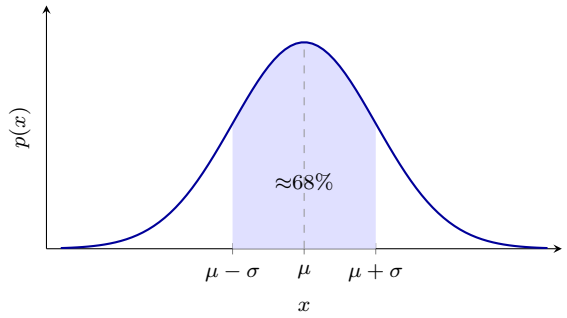
- μ (the *mean*) — the **center**: the position of the peak, the value samples come out around. Changing μ slides the bell along the axis; the shape stays.

- σ (the *standard deviation*) — the **width**: how far a typical sample lands from the center. Small σ: values cluster tightly around μ. Large σ: they scatter widely. Doubling σ makes the bell twice as wide and half as tall (the total area stays 1).

Both are computed from data $x_1, \ldots, x_N$ by plain averaging:

$$\mu = \frac{1}{N}\sum_{n=1}^{N} x_n, \qquad \sigma = \sqrt{\frac{1}{N}\sum_{n=1}^{N}(x_n - \mu)^2}$$

— the average value, and the square root of the average squared distance from it. Example: samples $1, 3, 5, 7$ give $\mu = 4$ and $\sigma = \sqrt{\frac{9+1+1+9}{4}} = \sqrt{5} \approx 2.24$. The next section derives both formulas from first principles (μ = expectation, σ^2 = variance).



The shaded area between $\mu-\sigma$ and $\mu+\sigma$ holds about 68% of the probability. In ML the Gaussian is the default source of randomness: weights are initialized from a Gaussian, diffusion models corrupt images with Gaussian noise, and measurement noise is modeled as Gaussian unless there is a reason not to.

6.5 Using a distribution: pick the top, or sample

Given a distribution, a model can act in two ways: take the *most probable* outcome ($\arg\max_i p_i$ — deterministic, always "cat"), or *sample* — draw an outcome at random in proportion to the probabilities, so "dog" comes up 20% of the time. Language models *sample*: that is why the same prompt produces different continuations, and the temperature and top-p controls of text generation are ways of reshaping the distribution before sampling from it.

6.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
X	a random variable: a quantity whose value is drawn, not fixed
$P(X = i)$	the probability that X comes out as outcome i
K	the number of possible outcomes (classes, tokens, …)
p_i / $p(x)$	probability of outcome i (discrete) / density at x (continuous)
$z,\ z_i$	the logits: raw pre-softmax scores (unrelated to §4's z)
$\mathcal{N}(\mu, \sigma^2)$	Gaussian; μ = mean (center of the bell), σ = standard deviation (width)
$\arg\max_i p_i$	the index i of the largest p_i

7 Expectation and variance

A distribution (§6) holds many numbers. Two summaries compress it into something usable: the *expectation* (where the outcomes are centered) and the *variance* (how widely they spread). Both matter to ML for one deep reason, kept for the end of the section: **a loss is an expectation**.

7.1 Expectation: the probability-weighted average

Let X take the value x_i with probability p_i $(i = 1, \dots, K)$. The *expectation* (also *mean*) of X is

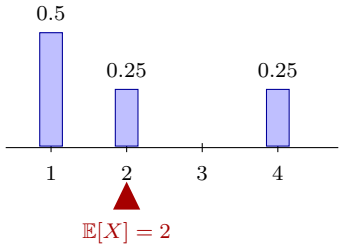
$$\mathbb{E}[X] \;=\; \sum_{i=1}^{K} p_i \, x_i$$

— each value, weighted by how often it happens. Despite the name, it is *not* "the value you expect": a fair die has $\mathbb{E}[X] = \frac{1}{6}(1+2+3+4+5+6) = 3.5$, a value the die cannot even show. It is the *long-run average*: roll many times, average the results, and the average settles toward 3.5.

Worked, and drawn. Values $1, 2, 4$ with probabilities $0.5, 0.25, 0.25$:

$$\mathbb{E}[X] = 1 \cdot 0.5 + 2 \cdot 0.25 + 4 \cdot 0.25 = 0.5 + 0.5 + 1 = 2.$$

Picture the probabilities as weights sitting on a number line; the expectation is the point where the line *balances*:



Note that the balance point need not carry a tall bar: the heavy weight at 1 and the distant weight at 4 balance around 2.

Two facts are used constantly. *Linearity*: $\mathbb{E}[aX + b] = a\,\mathbb{E}[X] + b$, and $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ for *any* two random variables — averages add, no conditions attached. *Continuous case*: the sum becomes an integral, $\mathbb{E}[X] = \int x\, p(x)\, dx$, weighting each value by the density of §6.

7.2 Variance: the spread around the mean

Write $\mu = \mathbb{E}[X]$. The *variance* is the expected *squared* distance from the mean:

$$\mathrm{Var}[X] \;=\; \mathbb{E}\big[(X-\mu)^2\big] \;=\; \sum_{i=1}^{K} p_i \, (x_i - \mu)^2.$$

Why squared? Raw deviations average to zero by the definition of μ — the negatives cancel the positives — so we square to make every deviation count, and squaring is smooth, which matters once derivatives (§3) enter. The price is squared units; the *standard deviation* $\sigma = \sqrt{\mathrm{Var}[X]}$ takes the square root to restore them.

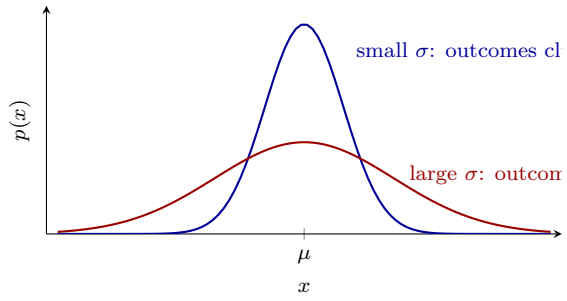
Worked, same example ($\mu = 2$):

$$\mathrm{Var}[X] = 0.5\,(1-2)^2 + 0.25\,(2-2)^2 + 0.25\,(4-2)^2 = 0.5 + 0 + 1 = 1.5, \qquad \sigma \approx 1.22.$$

A second route to the same number — the *shortcut formula* $\mathrm{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$:

$$\mathbb{E}[X^2] = 0.5 \cdot 1 + 0.25 \cdot 4 + 0.25 \cdot 16 = 5.5, \qquad 5.5 - 2^2 = 1.5.\checkmark$$

Same mean, different variance — the summaries answer different questions:



This confirms §6's preview: in $\mathcal{N}(\mu, \sigma^2)$, the parameter μ *is* the Gaussian's expectation and σ^2 *is* its variance — the center and width described there, now derived precisely.

7.3 Sample averages estimate expectations

The formulas above need the true probabilities p_i — which in practice nobody has. What you have is *samples*: N independent draws $X_1, \ldots, X_N$ of the same random variable. Their plain average estimates the expectation,

$$\frac{1}{N}\sum_{n=1}^{N} X_n \;\approx\; \mathbb{E}[X],$$

and the estimate sharpens as N grows (the *law of large numbers*); its own variance shrinks like $\mathrm{Var}[X]/N$, so four times as many samples halve the noise. This one approximation — *average over draws instead of summing over the unknown distribution* — is what lets machine learning pass from theory to data.

7.4 Why ML cares: the loss is an expectation

What training *wants* to minimize is the model's average wrongness over the true data distribution — an expectation:

$$L(\theta) = \mathbb{E}_{\text{data}}\big[\,\ell(\text{model's answer},\ \text{truth})\,\big].$$

That distribution is unknown, so training minimizes the sample-average stand-in — the loss over the N examples of the training set:

$$L_{\text{train}}(\theta) = \frac{1}{N}\sum_{n=1}^{N} \ell_n.$$

Three ideas of the next chapter fall out of this picture at once. *Generalization* is the question of how far the sample average sits from the true expectation. *Overfitting* is minimizing the sample average so aggressively that the gap widens. And *mini-batch training* replaces the full average by an average over a small random batch — a noisier estimate of the *same* expectation, cheap per step, with variance controlled by the batch size. Variance also returns inside neural networks: weight initialization schemes are designed precisely to keep the variance of each layer's outputs stable.

7.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$\mathbb{E}[X]$	expectation: the probability-weighted average $\sum_i p_i x_i$
μ	shorthand for $\mathbb{E}[X]$ (the mean)
$\mathrm{Var}[X]$	variance: $\mathbb{E}[(X-\mu)^2]$, the mean squared deviation
σ	standard deviation $\sqrt{\mathrm{Var}[X]}$ (same units as X)
$X_1, \ldots, X_N$	N independent draws (samples) of X; n indexes draws, i outcomes
ℓ, ℓ_n	the per-example loss, and its value on training example n

8 Conditional probability and Bayes' rule

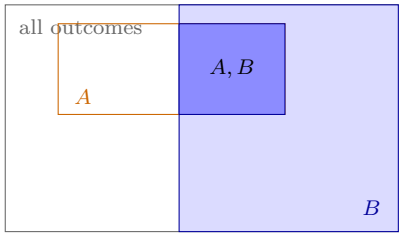
§6 treated one distribution at a time. Real questions come with *information attached*: what is the probability of A, *given that* B is known to have happened? That "given that" is the single most important construction in this chapter — a language model is, in its entirety, a machine for computing $P(\text{next token} \mid \text{previous tokens})$.

8.1 The conditioning bar

Write $P(A \mid B)$ — "the probability of A given B" — and $P(A, B)$ for the *joint* probability that both happen. The definition:

$$P(A \mid B) \;=\; \frac{P(A, B)}{P(B)}.$$

The idea behind the formula: learning that B happened *shrinks the world*. Outcomes outside B are no longer possible, so we keep only B's region and re-scale it to total probability 1 (that is the division by $P(B)$). Within the shrunken world, A's share is whatever overlap it had with B:



given B: only the blue region remains, and $P(A \mid B) = \dfrac{\text{overlap}}{\text{all of } B}$

8.2 Worked with real counts

Formulas about probability are easiest to trust as *counts*. Take 100 emails, classified two ways — spam or not, and containing the word "free" or not:

	contains "free"	no "free"	total
spam	12	8	20
not spam	4	76	80
total	16	84	100

Every probability is a ratio of two cells:

$$P(\text{spam}) = \tfrac{20}{100} = 0.2, \qquad P(\text{free} \mid \text{spam}) = \tfrac{12}{20} = 0.6, \qquad P(\text{spam} \mid \text{free}) = \tfrac{12}{16} = 0.75.$$

The last two read off different rows of the same overlap cell (12): given *spam*, divide by the spam row's total 20; given *"free"*, divide by the "free" column's total 16.

Note the asymmetry: $P(\text{free} \mid \text{spam}) = 0.6$ but $P(\text{spam} \mid \text{free}) = 0.75$. **The two directions are different questions with different answers** — confusing them is one of the most common reasoning errors with probability, inside ML and out.

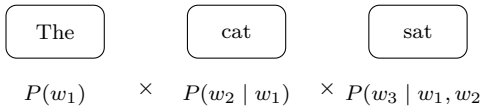
8.3 The product rule, and how language models factorize

Rearranging the definition gives the *product rule*:

$$P(A, B) \;=\; P(B)\,P(A \mid B) \;=\; P(A)\,P(B \mid A).$$

Applied repeatedly, it splits a *sequence* into one conditional per element. For tokens $w_1, w_2, \ldots, w_T$ (positions 1-based, per the convention):

$$P(w_1, w_2, \ldots, w_T) \;=\; \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}).$$



This factorization *is* the language model. Each factor is one categorical distribution over the vocabulary (§6), produced by a softmax, conditioned on everything before it; generating text is sampling from these conditionals one position at a time, feeding each drawn token back in as context. Nothing about the factorization is approximate — the product rule makes it exact.

8.4 Bayes' rule: reversing the direction

The product rule wrote $P(A, B)$ two ways. Set the two ways equal and divide by $P(B)$:

$$P(A \mid B) \;=\; \frac{P(B \mid A)\, P(A)}{P(B)}.$$

This is *Bayes' rule*, and its whole value is that it *reverses the conditioning direction*: it computes the direction you want from the direction you can measure. Each factor has a traditional name, worth knowing because the literature uses them constantly:

Term	Name	Plain reading
$P(A)$	prior	how plausible A was before seeing evidence
$P(B \mid A)$	likelihood	how well A explains the observed B
$P(B)$	evidence	how common the observation B is overall
$P(A \mid B)$	posterior	how plausible A is after seeing B

Check it against the table. The goal is $P(\text{spam} \mid \text{free})$; Bayes' rule assembles it from the other direction:

$$P(\text{spam} \mid \text{free}) = \frac{P(\text{free} \mid \text{spam})\, P(\text{spam})}{P(\text{free})} = \frac{0.6 \cdot 0.2}{0.16} = \frac{0.12}{0.16} = 0.75,$$

exactly the value counted directly from the table. ✓ (This direction-reversal is not a curiosity: the earliest practical spam filters were literally this computation, and the next section's maximum likelihood is what remains of Bayes when the prior is left out.)

Generality. Bayes' rule as written holds for *any* two events A, B in a system with *any* number of outcomes, whenever $P(B) > 0$. It does not require independence either — for independent events $P(A \mid B) = P(A)$, so there would be nothing to update. (Do not confuse *independent* — knowing one tells nothing, $P(A, B) = P(A)P(B)$ — with *mutually exclusive* — cannot both happen, $P(A, B) = 0$.)

The only step that needs more structure is *expanding the denominator*. If hypotheses $A_1, \ldots, A_K$ are mutually exclusive and cover all outcomes (a *partition*), then $P(B) = \sum_j P(B \mid A_j)\, P(A_j)$ — the *law of total probability* — giving the K-hypothesis Bayes' rule:

$$P(A_i \mid B) = \frac{P(B \mid A_i)\, P(A_i)}{\sum_{j=1}^{K} P(B \mid A_j)\, P(A_j)}.$$

Our table is the $K = 2$ case: $P(\text{free}) = 0.6 \cdot 0.2 + 0.05 \cdot 0.8 = 0.16$. ✓ With K classes this is exactly a classifier's posterior: likelihood × prior per class, normalized by the total.

8.5 Independence and "i.i.d."

A and B are *independent* when knowing one tells nothing about the other:

$$P(A \mid B) = P(A) \qquad \Longleftrightarrow \qquad P(A, B) = P(A)\, P(B).$$

The abbreviation *i.i.d.* — *independent and identically distributed* — describes draws that do not influence each other and all come from the same distribution. It is the standing assumption behind §7's sample averages, and the standard (if idealized) assumption about the examples in a training set. When it fails — consecutive video frames, text scraped twice — the effective amount of information is smaller than the number of examples suggests.

8.6 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$P(A \mid B)$	conditional probability: A's probability once B is known
$P(A, B)$	joint probability: both A and B happen
w_t	the token at position t (positions $1, \ldots, T$)
prior / posterior	$P(A)$ before evidence / $P(A \mid B)$ after evidence
likelihood / evidence	$P(B \mid A)$ / $P(B)$ in Bayes' rule
$A_1, \ldots, A_K$	a partition: mutually exclusive hypotheses covering every outcome
mutually exclusive	cannot both happen ($P(A, B) = 0$) — not the same as independent
i.i.d.	independent and identically distributed draws

9 Maximum likelihood estimation

Every section so far worked with distributions whose numbers were already given. Learning runs the other way: you hold *data* and must choose the distribution's *parameters*. A parameter is a number in the model's formula that the formula leaves free — setting it selects which specific distribution you have: the heads-probability p of a coin, the μ and σ of a Gaussian (§6), later the millions of weights of a neural network. The symbol θ collectively stands for all of a model's parameters at once. Maximum likelihood estimation (MLE) is the standard principle for choosing them, and it is one sentence long: **choose the parameters that make the observed data most probable.**

9.1 Likelihood: the same formula, read backwards

Suppose a model with parameters θ assigns the observed dataset a probability $P(\text{data} \mid \theta)$ — the *likelihood* from Bayes' table in §8. Now fix the data (it already happened) and read the expression as a function *of* θ:

$$\mathcal{L}(\theta) = P(\text{data} \mid \theta).$$

Same formula, opposite reading: as a function of data it is a probability; as a function of θ it scores *how well each candidate parameter explains what was seen.* MLE picks the best-scoring candidate:

$$\hat{\theta} = \arg\max_{\theta} \ \mathcal{L}(\theta)$$

— the hat $\hat{}$ marks "an estimate computed from data", as opposed to the unknown true value. (Note $\mathcal{L}$ is *not* the loss L of §7; the exact relation between the two arrives below.)

9.2 First worked example: which coin am I holding?

There are three known coins:

$$p_A = 0.3, \qquad p_B = 0.5, \qquad p_C = 0.8,$$

where p_θ is the probability that coin θ lands heads. Each coin has its own distribution; for C it is $P(\text{H}) = 0.8$, $P(\text{T}) = 0.2$, and it sums to 1. The three numbers p_A, p_B, p_C belong to three *different* coins, so they need not sum to 1.

Someone hands you one of the three coins without telling you which. The parameter to estimate is the identity of your coin: $\theta \in \{A, B, C\}$.

To find out, you flip the coin three times. Flip outcomes are random; you only observe what comes out. In this run, what comes out is H, H, T. This sequence is your data.

For each candidate, compute how probable the observed H, H, T would be *if the coin were that one.* That number is $P(\text{H, H, T} \mid \theta)$, the conditional of §8. The flips are independent, so multiply one factor per flip: p_θ for each H, and $1 - p_\theta$ for the T:

Candidate θ	$\mathcal{L}(\theta) = P(\text{H, H, T} \mid \theta)$	Value
A	$0.3 \cdot 0.3 \cdot 0.7$	0.063
B	$0.5 \cdot 0.5 \cdot 0.5$	0.125
C	$0.8 \cdot 0.8 \cdot 0.2$	**0.128**

Coin C gives the observed data the highest probability ($0.128 > 0.125 > 0.063$). So MLE answers $\hat{\theta} = C$: *your coin is most plausibly the one with heads-probability* 0.8. The estimate is $\hat{p} = 0.8$. And

the estimate follows the data: in a different run the random flips might land T, T, H — for *that* observed sequence, the same table computation would make coin A the winner ($0.7{\cdot}0.7{\cdot}0.3 = 0.147$). The next subsection asks the same question with a continuous parameter $p \in [0, 1]$, so the maximum is found by calculus instead of by comparing rows.

9.3 Second worked example: the biased coin

Now remove the three-item menu: the parameter is a *continuous* heads-probability $p \in [0, 1]$. Flip the coin four times; the outcomes come out H, H, T, H — $h = 3$ heads in $N = 4$ flips. The flips are i.i.d. (§8), so probabilities multiply:

$$\mathcal{L}(p) = p \cdot p \cdot (1-p) \cdot p = p^3(1-p).$$

Maximize by the rules of §3: expand $\mathcal{L}(p) = p^3 - p^4$, differentiate (power rule) and set to zero:

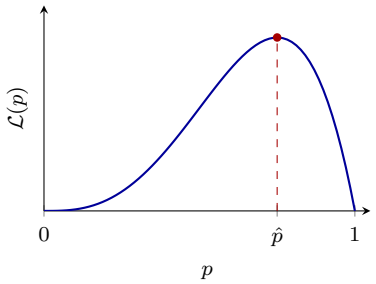
$$\mathcal{L}'(p) = 3p^2 - 4p^3 = p^2\,(3 - 4p) = 0 \quad \Longrightarrow \quad \hat{p} = \tfrac{3}{4}$$

(the root $p = 0$ is a minimum — it gives the data probability zero).

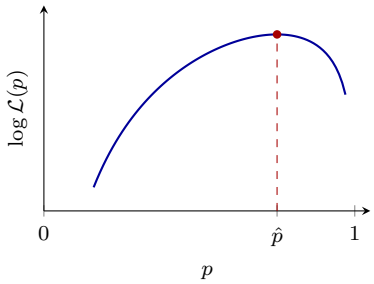
The estimate $\hat{p} = \frac{3}{4}$ is the best single guess of the coin's unknown heads-probability: among all candidate values $p \in [0, 1]$, the value 0.75 gives the observed sequence H, H, T, H the highest probability. The working model of the coin becomes "lands heads with probability 0.75" — the number used to predict future flips. The MLE also equals the *observed frequency* $h/N = 3/4$; in general, h heads in N flips give $\hat{p} = h/N$.

9.4 The log trick: products become sums

Real datasets have millions of factors, and a product of millions of numbers below 1 is unmanageable — numerically (it underflows to zero) and algebraically (its derivative needs the product rule a million times). The standard move: maximize $\log \mathcal{L}$ instead. This is allowed because log is strictly increasing, so it *reorders nothing* — whichever θ maximizes $\mathcal{L}$ also maximizes $\log \mathcal{L}$:



likelihood $p^3(1-p)$



log-likelihood $3\log p + \log(1-p)$

Different curves, *same maximizer* $\hat{p} = 0.75$. And the algebra gets easier. For the coin,

$$\log \mathcal{L}(p) = 3\log p + \log(1-p),$$

and using $(\log x)' = 1/x$ (§3) plus the chain rule on $\log(1-p)$ (inner derivative -1, §4):

$$\frac{d}{dp}\log \mathcal{L} = \frac{3}{p} - \frac{1}{1-p} = 0 \quad \Longrightarrow \quad 3(1-p) = p \quad \Longrightarrow \quad \hat{p} = \tfrac{3}{4}.\checkmark$$

For an i.i.d. dataset $x_1, \ldots, x_N$ the pattern is always the same:

$$\log \mathcal{L}(\theta) = \log \prod_{n=1}^{N} p(x_n \mid \theta) = \sum_{n=1}^{N} \log p(x_n \mid \theta),$$

one clean term per example. Frameworks *minimize*, so in practice one minimizes the *negative* log-likelihood (NLL) — the same sum, sign-flipped and averaged over the N examples:

$$L_{\text{NLL}}(\theta) \;=\; -\frac{1}{N} \sum_{n=1}^{N} \log p(x_n \mid \theta).$$

This is precisely a loss $L(\theta)$ in the sense of §7; maximizing likelihood and minimizing the NLL loss are the same act.

9.5 Second payoff: least squares is Gaussian MLE

Take N real-valued observations $x_1, \ldots, x_N$, and assume each one is a draw from the same Gaussian with unknown center:

$$x_n \sim \mathcal{N}(\mu, \sigma^2), \qquad n = 1, \ldots, N.$$

The symbol $\sim$ reads "is distributed as" — a draw from that distribution (§6). An equivalent reading: each observation is the unknown true value plus a random error,

$$x_n = \mu + \varepsilon_n, \qquad \varepsilon_n \sim \mathcal{N}(0, \sigma^2),$$

where the error (the *noise*) is Gaussian around zero — this assumption is called a *Gaussian noise model*. Here σ is treated as known and fixed; the parameter to estimate is μ. The Gaussian density of §6 assigns each observation

$$p(x_n \mid \mu) = \frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-\frac{(x_n-\mu)^2}{2\sigma^2}}.$$

The bar is §8's "given", applied to a *parameter* rather than an event: $p(x_n \mid \mu)$ reads "the density of x_n, given that the center is μ" — the same conditioning as in $P(\text{data} \mid \theta)$ throughout this section. (Some texts write $p(x_n; \mu)$, "with parameter μ", for exactly this.) The observations are i.i.d. (§8), so the likelihood of the whole dataset is the product of the per-observation densities:

$$\mathcal{L}(\mu) \;=\; \prod_{n=1}^{N} p(x_n \mid \mu).$$

Taking the log turns the product into a sum (the log trick above); inside each term the log cancels the e, leaving the exponent, and all terms not involving μ are collected into a constant:

$$\log \mathcal{L}(\mu) = -\frac{1}{2\sigma^2} \sum_{n=1}^{N} (x_n - \mu)^2 \;+\; \text{const.}$$

Maximizing this is exactly *minimizing* $\sum_n (x_n - \mu)^2$ — the sum of squared errors. Setting the derivative to zero (chain rule: $\frac{d}{d\mu}(x_n - \mu)^2 = -2(x_n - \mu)$):

$$\sum_{n=1}^{N} (x_n - \mu) = 0 \quad \Longrightarrow \quad \hat{\mu} = \frac{1}{N} \sum_{n=1}^{N} x_n,$$

the sample mean of §7.

Naming the pieces just used: the quantity $\frac{1}{N}\sum_n (x_n - \mu)^2$ — the average squared gap between the model's value and the observed one — is the *mean-squared-error (MSE) loss*, the standard loss for predicting numbers (regression). The *log-likelihood* is $\log \mathcal{L}$ from the log trick. The *Gaussian noise model* is the assumption $x_n = \mu + \varepsilon_n$ above. The general lesson: **the MSE loss is not an arbitrary choice — it is derived: assume Gaussian noise and maximize likelihood, and minimizing MSE is what falls out** (the log-likelihood, sign-flipped and divided by N). Every later use of MSE in machine learning is this section, applied.

9.6 MLE in real AI: training a language model

The biased coin above had a single parameter — its heads-probability p — and its maximization could be finished by hand: set the derivative $\frac{d}{dp}\log \mathcal{L} = \frac{3}{p} - \frac{1}{1-p}$ to zero and solve, giving $\hat{p} = \frac{3}{4}$. A real model — take a language model — runs the *same principle* with three changes.

The model. A neural network with parameters θ — billions of weights instead of one number p. Fed a context, it outputs a softmax distribution (§6) over the next token: $P(\text{next token} \mid \text{context}, \theta)$.

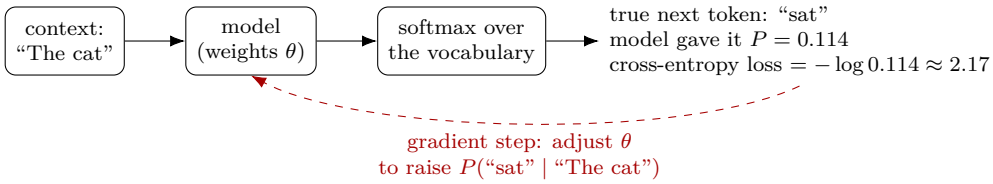
The likelihood. The data is a text corpus. By the factorization of §8, the probability the model assigns to the whole corpus is a product with one factor per position: the probability it gave to the token that *actually came next*. Taking the log turns it into a sum:

$$\log \mathcal{L}(\theta) = \sum_{\text{positions } t} \log P(\text{actual token at } t \mid \text{its context}, \theta).$$

The training loss is the NLL of this — in every framework it is literally named the *cross-entropy loss* (next section).

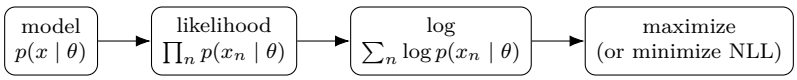
The maximization. With billions of parameters there is no "set the derivative to zero and solve". Instead, gradient descent on the NLL: compute $\partial\,\text{loss}/\partial\theta_i$ for every weight (that is *backpropagation*), nudge every weight a small step, repeat millions of times — each step on a random mini-batch, a cheap sample-average estimate of the full sum (§7). Training never reaches *the* maximum; it climbs as far as data and compute allow.

One position, concretely:



That adjustment, repeated over trillions of positions, *is* pretraining. Every weight update of a GPT-style model is one step of maximum likelihood.

9.7 The recipe, and two connections



Two connections to keep. *Backwards to Bayes* (§8): the posterior is likelihood × prior, normalized; MLE is what remains when the prior is flat — pure likelihood, no initial preference among parameters. *Forwards to cross-entropy*: for a classifier, the NLL of the true labels is exactly the cross-entropy loss — the next section builds that bridge.

9.8 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
θ	all of a model's parameters: the free numbers in its formula
$\mathcal{L}(\theta)$	likelihood: $P(\text{data} \mid \theta)$ read as a function of θ (not the loss L of §7; loss = NLL = $-\frac{1}{N}\log\mathcal{L}$)
$\hat{\theta},\ \hat{p},\ \hat{\mu}$	estimates computed from data (the hat marks an estimate)
p_A, p_B, p_C	heads-probabilities of the three candidate coins
$h,\ N$	number of heads, number of flips (or examples)
$\log\mathcal{L}$	log-likelihood: same maximizer, sum instead of product
$x_n \sim \mathcal{N}(\mu, \sigma^2)$	$\sim$ = "is distributed as": x_n is a draw from that Gaussian
ε_n	the noise: the random error in observation n, $x_n = \mu + \varepsilon_n$
NLL	negative log-likelihood: $L_{\text{NLL}} = -\frac{1}{N}\sum_n \log p(x_n \mid \theta)$

10 Entropy, cross-entropy, and KL divergence

One idea generates this whole section: the *surprise* of an outcome. Entropy is average surprise; cross-entropy is average surprise when probabilities come from the wrong distribution; KL divergence is the gap between the two. Cross-entropy is the standard training loss of deep learning — the ★ this chapter has been building toward.

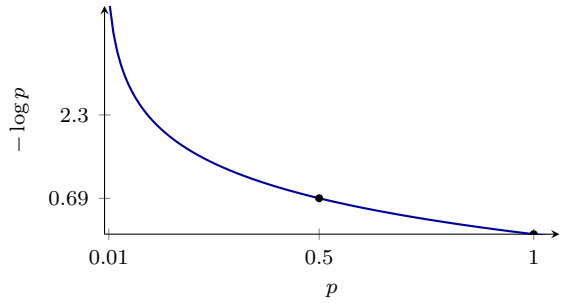
10.1 Surprise: $-\log p$

An outcome that had probability p carries surprise

$$s(p) = -\log p$$

(natural log, as in §9; measured in *nats*. With $\log_2$ the unit is *bits* — same idea, different scale). This is the only function that behaves like surprise should:

- certain events do not surprise: $s(1) = -\log 1 = 0$;
- the rarer the outcome, the larger the surprise: $-\log p$ grows as p falls (to ∞ as $p \to 0$);
- surprises of independent events *add*: probabilities multiply (§8), and $-\log$ turns products into sums (§9).



Worked values: $s(1) = 0$; $s(0.5) = \log 2 \approx 0.693$; $s(0.114) \approx 2.17$; $s(0.01) \approx 4.61$.

10.2 Entropy: expected surprise

The *entropy* of a distribution $p = (p_1, \ldots, p_K)$ is the expected surprise (§7) of a draw from it:

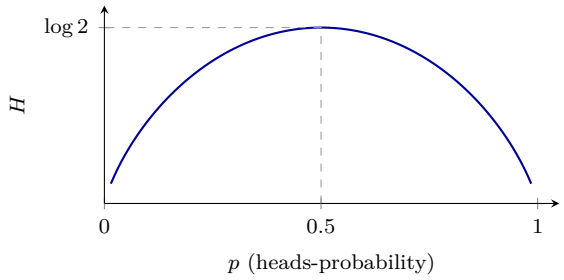
$$H(p) \;=\; \mathbb{E}[\,s\,] \;=\; -\sum_{i=1}^{K} p_i \log p_i.$$

It measures the distribution's inherent uncertainty: how surprised one is *on average*, knowing the true probabilities.

Worked (three coins):

Coin	$H(p)$	Reading
certain, $(1, 0)$	0	no uncertainty at all
biased, $(0.9, 0.1)$	$0.9 \cdot 0.105 + 0.1 \cdot 2.303 = 0.325$	mostly predictable
fair, $(0.5, 0.5)$	$2 \cdot 0.5 \cdot 0.693 = 0.693$	maximal for two outcomes

For a coin with heads-probability p, the whole curve $H = -p \log p - (1-p)\log(1-p)$ confirms the pattern — zero at the certain edges, peak at fair:



10.3 Cross-entropy: surprise under the wrong distribution

Three definitions:

- The *true distribution* p is the list of rates at which the outcomes actually happen. For a fair coin, $p = (p_{\mathrm{H}}, p_{\mathrm{T}}) = (0.5, 0.5)$: heads happens 50% of the time, tails happens 50% of the time.
- The *model's distribution* q is the list of probabilities that the model stores. Suppose the model stores $q = (q_{\mathrm{H}}, q_{\mathrm{T}}) = (0.9, 0.1)$. These stored numbers are wrong — they differ from the real rates. (The textbook phrase "the model believes q" means exactly: the model stores q.)
- The *cross-entropy* $H(p, q)$ is the average surprise of a model that stores q, measured over outcomes that happen at the rates p.

The computation has two steps:

1. The surprise of each single outcome, computed from the *stored* numbers (surprise $= -\log$ of the stored probability):

$$\text{heads: } -\log q_{\mathrm{H}} = -\log 0.9 = 0.105, \qquad \text{tails: } -\log q_{\mathrm{T}} = -\log 0.1 = 2.303.$$

2. The *weighted average* of these surprises. A weighted average multiplies each surprise by a weight and adds; because the weights sum to 1, no division is needed — the division of a plain average is built into the weights. Here the weights are the real rates $p_{\mathrm{H}}, p_{\mathrm{T}}$ (this is exactly §7's expectation, applied to the surprise):

$$H(p, q) = p_{\mathrm{H}} \cdot 0.105 + p_{\mathrm{T}} \cdot 2.303 = 0.5 \cdot 0.105 + 0.5 \cdot 2.303 = 1.204.$$

   For this fair coin the weights happen to be 0.5 each, so the result coincides with the plain average $\frac{0.105+2.303}{2}$; with unequal rates — say $p = (0.9, 0.1)$ — the heads term would count nine times as heavily, and the plain average would be wrong.

The same two steps, written for K outcomes, give the general formula:

$$H(p, q) = \sum_{i=1}^{K} \underbrace{p_i}_{\text{real rate}} \underbrace{(-\log q_i)}_{\text{surprise from } q_i} = -\sum_{i=1}^{K} p_i \log q_i.$$

Compare with a model that stores the *correct* numbers, $q = p = (0.5, 0.5)$. Its surprise for either outcome is $-\log 0.5 = 0.693$, so its weighted average is

$$0.5 \cdot 0.693 + 0.5 \cdot 0.693 = 0.693$$

— exactly the fair coin's entropy $H(p)$ from the table above. So the two models compare as: correct numbers, average surprise 0.693; wrong numbers, average surprise 1.204. This is general: $H(p, q) \geq H(p)$ always, with equality exactly when $q = p$. (The gap between the two gets its own name below.)

The purpose of this construction: in real machine learning, the true rates p are *unknown* — the toy coin's 0.5 is visible only because the example was built that way. The model's q is a *guess* of p, and the goal of training is to push the guess toward the unknown truth. Cross-entropy is what makes that goal workable: it can be computed from observed outcomes alone (the ML case, next), and since $H(p, q) \geq H(p)$ with equality only at $q = p$, driving it down drives q toward p.

The ML case. In training, the truth for one example is an observed label — a distribution putting probability 1 on the correct class (*one-hot*). With $p = (0, \ldots, 1, \ldots, 0)$, all terms of the sum vanish except the true class i^*, and

$$H(p, q) = -\log q_{i^*}$$

— the surprise the model assigned to the correct answer. This is exactly the per-example NLL of §9, which is what frameworks call the *cross-entropy loss*:

Example	True class	Model's q_{i^*}	Loss $-\log q_{i^*}$
§6's classifier	cat	0.7	0.357
the same, if the truth were bird	bird	0.1	2.303
§9's language model	"sat"	0.114	2.17

A confidently correct prediction yields a small loss; a confidently wrong one yields a large loss — so this loss pushes the model to place probability on what actually happens. (ML texts often say "cost" for the same quantity: the loss function is also called the *cost function*; money is never meant.) Two levels must not be mixed. *Per example*, the cross-entropy $-\log q_{i^*}$ carries no $\frac{1}{N}$ — it is one example's surprise, and it equals that example's term in §9's log-likelihood sum. *Per dataset*, the cross-entropy loss is the average of these per-example values over the N training examples,

$$\frac{1}{N}\sum_{n=1}^{N} H\big(p^{(n)},\, q^{(n)}\big) \;=\; \frac{1}{N}\sum_{n=1}^{N}\big(-\log q_{i^*}^{(n)}\big) \;=\; L_{\mathrm{NLL}}(\theta)$$

($p^{(n)}$ is example n's one-hot truth, $q^{(n)}$ the model's output distribution for example n) — exactly §9's NLL, with the $\frac{1}{N}$ appearing at the dataset level in both. (Dividing by the constant N changes no minimizer in any case.) The whole connection fits in one sentence: **minimizing cross-entropy equals maximizing likelihood, because the dataset's cross-entropy loss is the negative log-likelihood of the observed labels, averaged over the examples.**

10.4 KL divergence: the price of the wrong belief

Subtracting the unavoidable part $H(p)$ from the cross-entropy leaves the *extra* surprise caused purely by using the stored q instead of the true p — the *Kullback–Leibler (KL) divergence*:

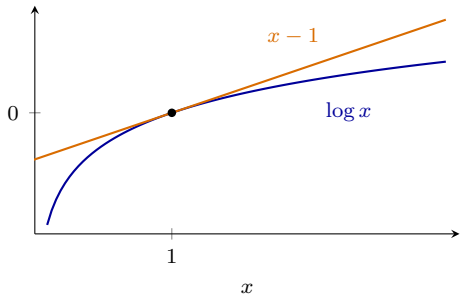
$$\begin{aligned} D_{\mathrm{KL}}(p\,\|\,q) &= H(p,q) - H(p) \\ &= \Big(-\sum_{i=1}^{K} p_i \log q_i\Big) - \Big(-\sum_{i=1}^{K} p_i \log p_i\Big) && \text{(insert both definitions)} \\ &= \sum_{i=1}^{K} p_i\big(\log p_i - \log q_i\big) && \text{(merge into one sum, factor out } p_i) \\ &= \sum_{i=1}^{K} p_i \log \frac{p_i}{q_i} && (\log a - \log b = \log \tfrac{a}{b}). \end{aligned}$$

It is ≥ 0, and $= 0$ exactly when $q = p$ — a measure of how far q is from p.

Why it cannot be negative. The proof needs one fact about the logarithm:

$$\log x \;\leq\; x - 1 \qquad \text{for all } x > 0, \text{ with equality only at } x = 1$$

— the straight line $x - 1$ touches the curve $\log x$ at $x = 1$ and lies above it everywhere else:



Now flip the sign of D_{KL} (flipping the fraction inside a log flips the sign: $\log \frac{p}{q} = -\log \frac{q}{p}$) and apply the fact with $x = q_i/p_i$:

$$-D_{\mathrm{KL}}(p\,\|\,q) = \sum_{i=1}^{K} p_i \log \frac{q_i}{p_i} \;\leq\; \sum_{i=1}^{K} p_i\Big(\frac{q_i}{p_i} - 1\Big) = \sum_{i=1}^{K} q_i - \sum_{i=1}^{K} p_i = 1 - 1 = 0,$$

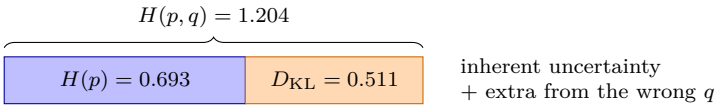
where the last step uses that both p and q are distributions, so each sums to 1 (§6). Therefore $-D_{\mathrm{KL}} \leq 0$, that is, $D_{\mathrm{KL}} \geq 0$. Equality requires every ratio q_i/p_i to sit at the touching point $x = 1$ — which is exactly $q = p$.

Worked (true $p = (0.5,\, 0.5)$, stored $q = (0.9,\, 0.1)$):

$$D_{\mathrm{KL}}(p\|q) = 0.5 \log \tfrac{0.5}{0.9} + 0.5 \log \tfrac{0.5}{0.1} = 0.5(-0.588) + 0.5(1.609) = 0.511,$$

and the three quantities fit together exactly as claimed:

$$H(p,q) = 0.5 \cdot 2.303 + 0.5 \cdot 0.105 = 1.204 = \underbrace{0.693}_{H(p)} + \underbrace{0.511}_{D_{\mathrm{KL}}}:$$



Two properties are worth remembering. *Asymmetry*: reversing the roles gives

$$D_{\mathrm{KL}}(q \,\|\, p) = 0.9 \log \tfrac{0.9}{0.5} + 0.1 \log \tfrac{0.1}{0.5} = 0.368 \;\neq\; 0.511,$$

so KL is not a distance — the direction matters. *Minimizing cross-entropy minimizes KL*: in the identity $H(p,q) = H(p) + D_{\mathrm{KL}}(p \,\|\, q)$, the term $H(p)$ is a constant — it depends only on the true rates, and training can only change q. So the only way training can lower the cross-entropy is by lowering D_{KL}, and the two minimizations select the same q: training drives the model's distribution toward the true one — the formal version of the purpose paragraph in the previous subsection.

KL returns constantly in machine learning: RLHF — the preference-tuning stage of language models — penalizes D_{KL} between the tuned model and the pretrained one so optimization does not drift too far, and VAEs — a family of generative models — use it to keep their latent space samplable.

10.5 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
$s(p) = -\log p$	surprise of an outcome that had probability p (in nats)
$H(p)$	entropy: expected surprise under the true distribution
q	the model's believed distribution (vs. the true p)
$H(p,q)$	cross-entropy: frequencies from p, surprise from q
i^*	the index of the true class; one-hot p has $p_{i^*} = 1$
$D_{\mathrm{KL}}(p\|q)$	KL divergence: $H(p,q) - H(p)$, the extra surprise from believing q
one-hot	a distribution with probability 1 on a single outcome